Modern İslamî Yaşam Rehberi Platformlarında Teknik Mimari ve Mühendislik Standartları: Veri Yönetimi, Astronomik Hesaplamalar ve Ölçeklenebilir Sistem Tasarımı

Dijital çağda İslamî içerik sunan mobil platformlar, statik metin depolamasından öte, karmaşık astronomik hesaplamalar, gerçek zamanlı veri senkronizasyonu ve milyonlarca kullanıcıya eşzamanlı hizmet verebilen altyapılar gerektirmektedir. Kur'an-ı Kerim metinlerinin kutsiyeti ve doğruluğu, namaz vakitlerinin astronomik hassasiyeti ve kullanıcı etkileşimlerinin sürekliliği, bu tür projeleri teknik açıdan "kritik sistemler" kategorisine sokmaktadır. React Native ile geliştirilen mobil arayüzün, NestJS tabanlı sağlam bir backend ve PostgreSQL gibi güçlü bir veritabanı mimarisiyle entegrasyonu, modern mühendislik pratiklerinin en üst düzeyde uygulanmasını zorunlu kılar. Bu rapor, söz konusu projenin "Best Practice" standartlarında hayata geçirilmesi için gerekli olan veri mimarisi, hesaplama mantığı, bildirim sistemleri, çevrimdışı çalışma stratejileri ve yazılım mimarisi katmanlarını derinlemesine analiz etmektedir.

Veri Mimarisi ve İçerik Yönetimi: Kur'an, Hadis ve Kıssa Verilerinin PostgreSQL Üzerinde Optimizasyonu

Kur'an-ı Kerim metni, mealleri ve binlerce hadis/kıssa verisi, bir uygulamanın omurgasını oluşturur. Bu verilerin yalnızca depolanması değil, milisaniyeler düzeyinde aranabilir ve erişilebilir olması kullanıcı deneyimi için hayatidir. PostgreSQL, bu tür yoğun metin verilerini yönetmek için gelişmiş "Full-Text Search" (FTS) yetenekleri sunar.[1] Geleneksel LIKE sorguları, veri seti büyüdükçe performans kaybına yol açarken, FTS metni "tsvector" adı verilen özel bir veri tipine dönüştürerek sözcüklerin köklerini (lexemes) indeksler.[3]

Full-Text Search ve İndeksleme Stratejileri

Arapça ve Türkçe gibi morfolojik açıdan zengin dillerde arama yaparken, kelimenin eklerini temizleyip köküne ulaşmak (stemming) kritik bir adımdır.[3] PostgreSQL varsayılan olarak birçok Avrupa dilini desteklese de, Arapça ve Türkçe için özel konfigürasyonlar gereklidir.[4] Arapça arama motoru tasarımı için Hunspell formatındaki sözlük dosyaları (ar.dic ve ar.aff) veritabanı sunucusuna entegre edilmelidir.[4] Bu sayede, kullanıcı "yecri" kelimesini aradığında, veritabanı "ce-ra-ya" kökünden türeyen tüm varyasyonları bulabilir.[4]

Veri modellemesinde performansın sürdürülebilirliği için "önceden hesaplanmış sütunlar" (materialized columns) stratejisi benimsenmelidir.[7] Her arama sorgusunda metni çalışma anında

to_tsvector fonksiyonuyla işlemek yerine, bu vektörler kayıt oluşturulurken veya güncellenirken bir trigger yardımıyla özel bir sütuna saklanmalıdır.[7] Bu yaklaşım, sorgu süresini yüzlerce kat hızlandırabilir.[7] İndeksleme tarafında ise GIN (Generalized Inverted Index) kullanımı standarttır.[1] GIN indeksleri, her bir sözcüğü geçtiği tüm satırlarla eşleştirerek hızlı erişim sağlar.[1]

İndeks Türü	Avantajları	Dezavantajları	En Uygun Kullanım Senaryosu
GIN	Çok hızlı arama performansı, çoklu anahtar kelime desteği.[1]	İndeks oluşturma ve güncelleme yavaştır, depolama alanı fazladır.[8]	Kur'an metni ve mealler gibi nadir güncellenen sabit içerikler.[7]
GiST	Hızlı güncelleme, daha küçük indeks boyutu, yakınlık araması desteği.[1]	GIN'e göre daha yavaş arama performansı.[8]	Sık güncellenen kullanıcı yorumları veya günlük paylaşımlar.[1]
Trigram	Yazım hatalarını tolere eder (fuzzy search), kısmi kelime eşleşmesi sağlar.[4]	Çok büyük veri setlerinde CPU kullanımı artabilir.[4]	Kullanıcıların arama kutusuna yazdığı hatalı sorguların düzeltilmesi.[4]

Sequelize ile Büyük Veri Setlerinin Yönetimi

Sequelize ORM katmanında, milyonlarca satırı yönetirken bellek tüketimini kontrol altında tutmak gerekir. findAll gibi metodlar, dönen sonuçları Sequelize nesnelerine (instance) dönüştürürken ciddi bir overhead yaratır.[9] Bu noktada { raw: true } opsiyonu kullanılarak verilerin ham JSON objesi olarak alınması sağlanmalıdır.[9] Ayrıca, findAndCountAll metodunun büyük veritabanlarında JOIN işlemleriyle birleştiğinde yavaşladığı ve yanlış sayımlar ürettiği bilinmektedir.[10] Bu sorunu aşmak için sayım ve veri getirme işlemlerini iki ayrı sorguya bölmek veya ham SQL sorgularıyla COUNT(*) optimizasyonu yapmak daha ölçeklenebilir bir yaklaşımdır.[10]

Namaz Vakti Mantığı: Astronomik Hesaplamalar ve Diyanet Standartları

Namaz vakitlerinin belirlenmesi, Güneş'in ufuk çizgisine göre olan konumunun matematiksel olarak hesaplanmasına dayanır. Bir mühendislik perspektifiyle, bu işlem iki aşamadan oluşur: astronomik temel hesaplamalar ve bölgesel/mezhepsel düzeltmeler (temkin süreleri). Projede

Aladhan gibi harici API'lar yerine NestJS tarafında yerel hesaplama kütüphanelerinin (örneğin Adhan.js veya PrayTimes türevleri) kullanılması, hem offline desteği hem de API maliyetlerinin düşürülmesi açısından tercih edilmelidir.[11]

Astronomik Hesaplama Parametreleri ve Diyanet Mantığı

Dünya genelinde farklı kurumlar (MWL, ISNA, Mısır Heyeti, Diyanet) farklı güneş açılarını baz almaktadır.[12] Türkiye ve Balkanlar özelinde Diyanet İşleri Başkanlığı'nın (DİB) standartları esastır. DİB hesaplamalarında en kritik parametreler İmsak ve Yatsı vakitleridir. Geleneksel olarak İmsak için güneşin ufkun altında -19° olduğu an kabul edilse de, Diyanet 1983 yılından itibaren bu açıyı -18° olarak revize etmiştir.[11] Yatsı vakti için ise genellikle -17° kullanılmaktadır.[11]

Temkin süreleri, vakit hesaplamalarına eklenen veya çıkarılan güvenlik marjlarıdır.[14] Bu süreler; atmosferik kırılma, bulunulan yerin yüksekliği ve ufuk çizgisi arasındaki farkları kompanse etmek amacıyla kullanılır.[14] Diyanet, modern dönemde İmsak ve Yatsı vakitlerinden temkin sürelerini kaldırmış, sadece Güneş (doğuş) ve Akşam (batış) vakitlerinde zorunlu astronomik düzeltmeleri korumuştur.[14] Öğle vakti için ise güneşin tam tepede olduğu ana (Zeval) yaklaşık 7 dakikalık bir güvenlik kayması eklenmektedir.[11]

Mezhepsel Farklılıkların Koda Aktarılması

Özellikle İkindi namazı vaktinde Hanefi mezhebi ile diğer mezhepler (Şafii, Maliki, Hanbeli) arasında ciddi bir fark bulunmaktadır. "Asr-ı Evvel" (birinci gölge) ve "Asr-ı Sani" (ikinci gölge) olarak adlandırılan bu durum, cismin gölgesinin boyunun kendisine olan oranına göre belirlenir.[11] Kod tarafında bu mantık bir konfigürasyon parametresi olarak yönetilmeli ve kullanıcının tercihine göre dinamik olarak hesaplanmalıdır.

TypeScript

```
// Mezhepsel İkindi Vakti Mantığı (Örnek Mantık)
const getAsrTime = (location: Coords, method: 'standard' | 'hanafi') => {
  const shadowFactor = method === 'hanafi'? 2 : 1;
  return calculationLibrary.calculateAsr(location, shadowFactor);
};
```

Vakit	Diyanet Açı/Parametre	Notlar
İmsak	-18.0°.[11]	1983 öncesi -19.0° idi; şu

		an temkin içermez.[14]
Güneş	Astronomik Doğuş.[17]	Görünür doğuş için temkin düzeltmesi yapılır.[14]
Öğle	Zeval + 7 dk.[11]	Güneşin en tepede olduğu andan sonraki emniyet süresi.[11]
İkindi	Gölge Oranı: 1 ($Asr -$).[11]	Hanefi mezhebinde yaygın görüş 2 kat gölgedir ($Asr -$).[11]
Akşam	Astronomik Batış + Temkin.[14]	Ufuk inhitâtı ve kırılma payı eklenir.[14]
Yatsı	-17.0°.[11]	Batı ufkundaki kızıllığın kaybolma anı baz alınır.[14]

Gelişmiş Bildirim (Notification) Sistemi: Ölçeklenebilirlik ve Pil Optimizasyonu

Milyonlarca kullanıcıya günde beş defa tam vaktinde namaz hatırlatıcısı göndermek, backend altyapısı için devasa bir eşzamanlılık yükü oluşturur. Bu yükü yönetmenin en sağlıklı yolu, doğrudan bir Cron Job içinde bildirim göndermek yerine, bir mesaj kuyruğu (Message Queue) yapısı kullanmaktır.[18]

BullMQ ve Redis ile Dağıtık Kuyruk Mimarisi

NestJS tabanlı bir sistemde BullMQ, Redis üzerinde çalışan son derece güvenilir bir kuyruk kütüphanesidir.[18] Bildirim gönderim süreci şu adımlarla kurgulanmalıdır:

1. **Zamanlayıcı (Scheduler)**: NestJS Cron servisi her dakika çalışarak o dakika içinde bildirim alması gereken kullanıcı segmentlerini tespit eder.[22]
2. **Kuyruğa Ekleme (Producer)**: Bildirim alacak kullanıcıların verileri (token, mesaj içeriği) BullMQ kuyruğuna aktarılır. Bu işlem API'nin ana işleyişini bloklamaz.[18]
3. **Worker İşleyicileri (Consumer)**: Ayrı süreçlerde (sandboxed processes) çalışan worker'lar, kuyruktan işleri alarak FCM'e (Firebase Cloud Messaging) iletir.[18]
4. **Hız Sınırı (Throttling)**: FCM servisinin rate-limit'lerine takılmamak için saniyedeki bildirim gönderim hızı BullMQ üzerinden limitlenmelidir.[23]

Pil Tüketimi ve Mobil Taraf Optimizasyonu

Mobil tarafta pil ömrünü korumak için "Silent Push Notifications" (Veri Mesajları) stratejisi izlenmelidir. FCM üzerinden notification objesi yerine data objesi gönderilmelidir. Bu sayede uygulama arka planda uyandırılır, yerel namaz vakti verisiyle bildirim içeriği doğrulanır ve yalnızca gerekli durumlarda kullanıcıya gösterilir. Ancak, namaz vakitleri gibi önceden bilinen olaylar için en iyi pratik, her günün başında veya haftalık olarak cihazın yerel takvimine (iOS UNUserNotificationCenter veya Android AlarmManager) bildirimlerin "Local Notification" olarak kaydedilmesidir. Bu yaklaşım, internet bağlantısı olmasa dahi bildirimin çalışmasını sağlar ve sürekli ağ trafiğini keserek pil tüketimini minimize eder.

Offline-First Yaklaşımı: WatermelonDB ve NestJS Senkronizasyonu

Bir yaşam rehberi uygulamasının kullanıcıyı yarı yolda bırakmaması için internet bağlantısı bir zorunluluk değil, bir geliştirme (enhancement) olarak görülmelidir. Kullanıcının Kur'an okumaya devam edebilmesi ve kendi favori ayetlerine/notlarına erişebilmesi için cihazda güçlü bir yerel veritabanı bulunmalıdır.

WatermelonDB'nin Mimari Avantajları

React Native ekosisteminde SQLite üzerine kurulu olan WatermelonDB, büyük veri setlerinde (binlerce hadis/ayet) yüksek performans sunmak için "Lazy Loading" (Tembel Yükleme) mantığını kullanır.[25] Uygulama açılışında tüm veriyi belleğe yüklemek yerine, sadece ekranda o an gösterilmesi gerekenleri çeker.[25] Ayrıca, veritabanındaki bir değişiklik (örneğin bir ayetin favoriye eklenmesi), arayüzdeki bileşenleri otomatik olarak tetikleyen "Reactive" bir yapıya sahiptir.[25]

Senkronizasyon Protokolü Tasarımı

Senkronizasyon süreci, WatermelonDB'nin standart protokolü olan "Pull-Push" döngüsü üzerinden kurgulanmalıdır.[28]

1. **Pull (Çekme)**: Cihaz sunucuya lastPulledAt zaman damgasını gönderir. NestJS tarafı, veritabanında bu tarihten sonra değişen kayıtları tespit eder.[29]
2. **Push (İtme)**: Cihaz, çevrimdışıyken yaptığı değişiklikleri (yeni notlar, silinen favoriler) paketleyerek sunucuya gönderir.[31]
3. **Conflict Resolution (Çatışma Çözümü)**: Eğer aynı veri hem sunucuda hem cihazda değişmişse, genellikle "Cihazın Değişikliği Kazansın" (Client Wins) stratejisi uygulanır.[32]

Clean Architecture ve SOLID Prensipleri: NestJS Katmanlı Mimari

Yüksek veri akışı olan projelerde kodun sürdürülebilirliği, iş mantığının (Business Logic) dış dünyadan (Veritabanı, Framework, API) izole edilmesine bağlıdır. NestJS üzerinde Clean Architecture uygulamak için proje dört ana katmana bölünmelidir.[33]

- **Domain (Çekirdek) Katmanı**: Uygulamanın en kararlı kısmıdır. Ayet, Hadis, User gibi varlıklar (Entities) barındırır.[33]
- **Application (Uygulama) Katmanı**: Kullanım senaryolarını (Use Cases) barındırır (örn: MarkAyetAsReadUseCase).[33]
- **Infrastructure (Altyapı) Katmanı**: Veritabanı implementasyonları (Sequelize) ve harici servisler (FCM) burada yer alır.[38]
- **Presentation (Sunum) Katmanı**: HTTP Controller'ları ve DTO'lar buradadır.[35]

Kıble ve Pusula: Sensör Füzyonu ve Hassas Yön Bulma Algoritması

Kıble yönü, kullanıcının koordinatları ile Kabe'nin koordinatları (21.4225° K, 39.8262° D) arasındaki küresel mesafenin açısıdır. React Native üzerinde bu işlem sadece bir trigonometrik hesaplama değil, donanım sensörlerinin gürültüsünü temizleme işlemidir.

Hesaplama için kullanılacak temel formül şöyledir [36]:

$$\tan(\theta) = \frac{\sin(\Delta\lambda)}{\cos(\phi_1)\tan(\phi_2) - \sin(\phi_1)\cos(\Delta\lambda)}$$

Hassas bir Kıble pusulası için GPS verisi alınmalı, bulunulan konuma göre **Manyetik Deklinasyon** (Sapma Açısı) hesaplanmalı ve ivmeölçer/jiroskop verileriyle stabilizasyon sağlanmalıdır.[39]

Proje Yapım Aşamaları (Roadmap)

Projenin teknik karmaşıklığı ve veri yoğunluğu göz önüne alındığında, geliştirme süreci aşağıdaki fazlara ayrılmalıdır:

Faz 1: Keşif, Analiz ve Mimari Kurulum (Hafta 1-2)

- **Keşif**: Hedef kitlenin ihtiyaç analizi ve rakiplerin benchmarking süreci.
- **Mimari Kurulum**: NestJS altyapısının "Clean Architecture" standartlarında ayağa kaldırılması, Sequelize ve PostgreSQL bağlantılarının yapılması.[35]
- **Veri Modelleme**: Kur'an, meal ve hadis tablolarının UUID ve FTS (Full-Text Search) destekli tasarımı.

Faz 2: Core Backend ve Veri Entegrasyonu (Hafta 3-5)

- **İçerik Yükleme**: Büyük veri setlerinin (Kur'an ve Hadis) PostgreSQL'e bulk olarak aktarılması ve Arapça/Türkçe Hunspell indekslerinin oluşturulması.[17]
- **Hesaplama Motoru**: Astronomik namaz vakti hesaplama servisinin (Adhan.js entegrasyonu) Diyanet standartlarına göre yazılması.

- **API Geliştirme**: İçerik sağlayıcı RESTful servislerin implementasyonu.

Faz 3: Mobil Temeller ve Offline-First Katmanı (Hafta 6-8)

- **Mobil Başlangıç**: React Native projesinin kurulması ve temel UI bileşenlerinin (Dashboard vb.) tasarımı.
- **Local DB**: WatermelonDB entegrasyonu ve NestJS ile "Pull-Push" senkronizasyon protokolünün kurulması.[25]
- **Sensör Entegrasyonu**: Kıble pusulası için cihaz sensörlerine erişim ve azimut algoritmasının implementasyonu.[37]

Faz 4: Bildirim Sistemi ve Ölçeklendirme (Hafta 9-11)

- **Kuyruk Yönetimi**: Redis ve BullMQ kurulumu. Milyonlarca kullanıcıya bildirim gönderecek "Worker"ların konfigüre edilmesi.[20]
- **Push Notification**: FCM entegrasyonu ve "Silent Push" stratejisinin uygulanması.
- **Performans Testleri**: Veritabanı sorgu optimizasyonları ve yük testleri.

Faz 5: Kalite Güvence (QA) ve Yayın (Hafta 12+)

- **Testler**: Birim (Unit), entegrasyon ve kullanıcı kabul testleri (UAT). Sensör hassasiyetinin farklı cihazlarda doğrulanması.
- **Deployment**: App Store ve Google Play Console süreçlerinin yönetimi.
- **İzleme**: Post-launch döneminde hata izleme (Sentry vb.) ve kullanıcı geri bildirimlerine göre iyileştirmeler.

Sonuç: Mühendislik Standartlarının Sentezi

Bu projenin başarısı, veri mühendisliği ile dini hassasiyetlerin teknoloji ile doğru harmanlanmasına bağlıdır. PostgreSQL üzerindeki FTS optimizasyonundan, BullMQ tabanlı ölçeklenebilir bildirim sistemine; WatermelonDB'nin offline-first gücünden, Clean Architecture'ın sürdürülebilirliğine kadar her adım, modern yazılım geliştirme metodolojilerinin en üst seviyede uygulanmasını gerektirir. Önerilen 12 haftalık roadmap, projenin hem sağlam bir teknik temel üzerine oturmasını hem de kullanıcı ihtiyaçlarını en yüksek hassasiyetle karşılamasını hedeflemektedir.

Alıntılanan çalışmalar

1. Full-Text Search in PostgreSQL - ParadeDB, erişim tarihi Nisan 8, 2026, https://www.paradedb.com/learn/search-in-postgresql/full-text-search
2. Documentation: 18: 12.1. Introduction - PostgreSQL, erişim tarihi Nisan 8, 2026, https://www.postgresql.org/docs/current/textsearch-intro.html
3. PostgreSQL Full Text Search: The Definitive Guide - DbVisualizer, erişim tarihi Nisan 8, 2026, https://www.dbvis.com/thetable/postgresql-full-text-search-the-definitive-guide/

4. Full text search in postgres - Wassim Ben Jdida, erişim tarihi Nisan 8, 2026, https://wassimbj.github.io/blog/full-text-search-in-postgres
5. Building Full Arabic Text Search with PostgreSQL | by Ali Shiyyab - Medium, erişim tarihi Nisan 8, 2026, https://medium.com/@aliakefsh/building-full-arabic-text-search-with-postgresql-2431aa282707
6. Full text search in PostgreSQL - Totara Help, erişim tarihi Nisan 8, 2026, https://totara.help/docs/full-text-search-in-postgresql
7. Speeding Up PostgreSQL Full-Text Search with Persistent ..., erişim tarihi Nisan 8, 2026, https://danielabaron.me/blog/speed-up-pg-fts-with-persistent-ts-vectors/
8. How to Implement Full-Text Search in PostgreSQL - OneUptime, erişim tarihi Nisan 8, 2026, https://oneuptime.com/blog/post/2026-01-21-postgresql-full-text-search/view
9. SELECT Queries: Finder Methods - Sequelize, erişim tarihi Nisan 8, 2026, https://sequelize.org/docs/v7/querying/select-methods/
10. Sequelize's findAndCountAll is Lying to You | by Manish Khdl Sy - Medium, erişim tarihi Nisan 8, 2026, https://medium.com/@manish.khdl.sy/sequelizes-findandcountall-is-lying-to-you-1d89b2f20b9a
11. Add Diyanet İşleri Başkanlığı as a calculation method · Issue #23 ..., erişim tarihi Nisan 8, 2026, https://github.com/batoulapps/Adhan/issues/23
12. Prayer Times API - AlAdhan, erişim tarihi Nisan 8, 2026, https://aladhan.com/prayer-times-api
13. Prayer Time Calculation Methods - AlAdhan, erişim tarihi Nisan 8, 2026, https://aladhan.com/calculation-methods
14. Temkin Meselesi - İnsan ve Hayat Dergisi, erişim tarihi Nisan 8, 2026, https://insanvehayat.com/temkin-meselesi/
15. "Türkiye Takvîmi" olarak, Diyânet İşleri Başkanlığının 1982 ve ..., erişim tarihi Nisan 8, 2026, https://www.turktakvim.com/pdf/MuhimAciklama.TT.pdf
16. namaz vakitleri ve temkin hakkında sual ve cevaplar - VFIB Rheinau e. V., erişim tarihi Nisan 8, 2026, https://www.vfib-rheinau.de/wp-content/uploads/2020/04/2018-camlica-yayin-katalogu.pdf
17. İstanbul Namaz Vakitleri - Diyanet Haber, erişim tarihi Nisan 8, 2026, https://www.diyanethaber.com.tr/istanbul-namaz-vakitleri
18. BullMQ with NestJS: Top 4 Use Cases & Quick Tutorial - Dragonfly, erişim tarihi Nisan 8, 2026, https://www.dragonflydb.io/guides/bullmq-nestjs
19. Building a Scalable Notification System in JavaScript - Python in Plain English, erişim tarihi Nisan 8, 2026, https://python.plainenglish.io/building-a-scalable-notification-system-in-javascript-890c5177b07d
20. Decoupling Firebase Push Notification Logic with BullMQ - From ..., erişim tarihi Nisan 8, 2026, https://dev.to/sangwoo_rhie/decoupling-firebase-push-notification-logic-with-bullmq-from-synchronous-chaos-to-asynchronous-2khp/

21. Queues | NestJS - A progressive Node.js framework, erişim tarihi Nisan 8, 2026, https://docs.nestjs.com/techniques/queues
22. Building a Production-Ready Scheduled Push Notification System with NestJS Cron and Firebase - DEV Community, erişim tarihi Nisan 8, 2026, https://dev.to/sangwoo_rhie/building-a-production-ready-scheduled-push-notification-system-with-nestjs-cron-and-firebase-4fai
23. Decoupling Firebase Push Notification Logic with BullMQ - From Synchronous Chaos to Asynchronous Elegance - DEV Community, erişim tarihi Nisan 8, 2026, https://dev.to/sangwoo_rhie/decoupling-firebase-push-notification-logic-with-bullmq-from-synchronous-chaos-to-asynchronous-2khp
24. Best practices when sending FCM messages at scale - Firebase - Google, erişim tarihi Nisan 8, 2026, https://firebase.google.com/docs/cloud-messaging/scale-fcm
25. Building Offline-First React Native Apps: Why WatermelonDB Is the ..., erişim tarihi Nisan 8, 2026, https://medium.com/@pelumiogundipe905/building-offline-first-react-native-apps-why-watermelondb-is-the-right-choice-96e28e87687f
26. Offline-First Architecture with WatermelonDB and MongoDB | by Vineetdixit Vns - Medium, erişim tarihi Nisan 8, 2026, https://medium.com/@vineetdixit.vns/offline-first-architecture-with-watermelondb-and-mongodb-7efee1a09bb0
27. Offline-first React Native Apps with Expo, WatermelonDB, and Supabase, erişim tarihi Nisan 8, 2026, https://supabase.com/blog/react-native-offline-first-watermelon-db
28. Sync implementation details - WatermelonDB, erişim tarihi Nisan 8, 2026, https://watermelondb.dev/docs/Implementation/SyncImpl
29. Frontend | WatermelonDB, erişim tarihi Nisan 8, 2026, https://watermelondb.dev/docs/Sync/Frontend
30. Backend - WatermelonDB, erişim tarihi Nisan 8, 2026, https://watermelondb.dev/docs/Sync/Backend
31. Using WatermelonDB for offline data sync - LogRocket Blog, erişim tarihi Nisan 8, 2026, https://blog.logrocket.com/watermelondb-offline-data-sync/
32. How to Implement Offline-First Architecture in React Native - OneUptime, erişim tarihi Nisan 8, 2026, https://oneuptime.com/blog/post/2026-01-15-react-native-offline-architecture/view
33. The Clean Architecture Pattern for NestJS: Scalable APIs Made ..., erişim tarihi Nisan 8, 2026, https://medium.com/@bhagyarana80/the-clean-architecture-pattern-for-nestjs-scalable-apis-made-simple-68300f36932c
34. NestJS-DDD-DevOps - Andrea Acampora, erişim tarihi Nisan 8, 2026, https://andrea-acampora.github.io/nestjs-ddd-devops/
35. Building Enterprise-Grade NestJS Applications: A Clean Architecture Template - V. Checha, erişim tarihi Nisan 8, 2026, https://v-checha.medium.com/building-enterprise-grade-nestjs-applications-a-clean-architecture-template-ebcb6462c692

36. Implementation of the Spherical Trigonometry Algorithm in a Qibla Direction Application, erişim tarihi Nisan 8, 2026, https://www.researchgate.net/publication/399476690_Implementation_of_the_Spherical_Trigonometry_Algorithm_in_a_Qibla_Direction_Application
37. Implementation of the Spherical Trigonometry Algorithm in a ... - arXiv, erişim tarihi Nisan 8, 2026, https://arxiv.org/html/2601.00808
38. Repository Pattern in NestJS: Do It Right or Go Home - DEV Community, erişim tarihi Nisan 8, 2026, https://dev.to/adamthedeveloper/repository-pattern-in-nestjs-do-it-right-or-go-home-268f
39. qibla-compass 1.0.0 on npm - Libraries.io, erişim tarihi Nisan 8, 2026, https://libraries.io/npm/qibla-compass
40. Create REST-API with Clean Architecture in NestJS | by Arsy Opraza | Level Up Coding, erişim tarihi Nisan 8, 2026, https://levelup.gitconnected.com/create-rest-api-with-clean-architecture-in-nestjs-d01b58d3f3bc
41. Documentation: 18: 12.3. Controlling Text Search - PostgreSQL, erişim tarihi Nisan 8, 2026, https://www.postgresql.org/docs/current/textsearch-controls.html
42. mmuminovic/react-native-qibla-compass - GitHub, erişim tarihi Nisan 8, 2026, https://github.com/mmuminovic/react-native-qibla-compass